

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON (8 SEMANAS)

NIVEL 4 (INTEGRADOR / PROFESIONAL)

Curso 4: Taller Práctico & Proyecto Final Integrador

«Construcción de Soluciones Reales: Full-Stack Web con FastAPI + Streamlit, Chatbot Inteligente y BD SQL ACID»

Manual completo del curso compilado oficialmente. Incluye el desglose de las 8 clases semanales, diagramas de arquitectura, código fuente comentado, resolución de gotchas y bibliografía de estudio.

Instructor: William Rodríguez (Wisrovi)

Total Clases: 8 Semanas | Python: 3.10+ | Licencia: MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología: La Regla de la Bicicleta

Aprender a programar es pedalear en VS Code: la experimentación práctica diaria es la clave del dominio técnico.

Plan de Estudios de las 8 Semanas

Semana	Unidad Temática	Metáfora
CLASE 01	Clase 01: Arquitectura de Software y Planificación del Proyecto	«Diseñar los Planos de un Edificio Antes de Poner e...»
CLASE 02	Clase 02: Desarrollo del Backend: APIs RESTful con FastAPI	«FastAPI como un Centro Logístico de Alta Velocidad...»
CLASE 03	Clase 03: Persistencia de Datos: Modelado SQL y Transacciones ACID	«La Base de Datos como una Bóveda Acorazada para la...»
CLASE 04	Clase 04: Desarrollo del Frontend: Dashboards con Streamlit	«Streamlit como el Salón de Control Visual para tu ...»
CLASE 05	Clase 05: Integración del Motor de IA y Agentes en la App	«Conectar el Cerebro del Agente al Sistema Nervioso...»
CLASE 06	Clase 06: Testing Riguroso con Pytest, Mocks y Calidad	«Los Tests como el Control de Calidad y Pruebas de ...»
CLASE 07	Clase 07: Containerización Profesional con Docker y Compose	«Docker como Contenedores Estándar de Carga Marítim...»
CLASE 08	Clase 08: Despliegue en la Nube, CI/CD y Portafolio Final	«Lanzamiento a Producción y Presentación de tu Proy...»

Clase 01: Arquitectura de Software y Planificación del Proyecto

Un proyecto de software profesional comienza con una arquitectura sólida que garantiza escalabilidad y mantenimiento.

☀ Metáfora: Diseñar los Planos de un Edificio Antes de Poner el Primer Ladrillo

Diseñar el software es como dibujar los planos estructurales de una casa: define dónde irán las tuberías (APIs) y los cimientos (BD).

Principios Teóricos

Separación de responsabilidades: Frontend (Presentación), Backend (Lógica de Negocio) y Base de Datos (Persistencia).

Definición de contratos API-First mediante esquemas OpenAPI y modelos Pydantic.

⚡ Regla de Oro

Nunca empieces a codificar sin tener un diagrama de arquitectura y las entidades de datos definidas.

Arquitectura y Movimiento de Datos

Arquitectura general del sistema distribuido cliente-servidor.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Diseño de la capa de presentación (Streamlit UI / Web).	Vistas y componentes maquetados.
2. Proceso	Especificación de endpoints REST en FastAPI.	Contratos DTO / Pydantic listos.
3. Salida	Integración del motor de Inteligencia Artificial.	Pipeline de agentes conectado.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class ProyectoConfig(BaseModel):
    nombre_app: str = "Wisrovi Enterprise App"
    version: str = "1.0.0"
    debug: bool = False

class ItemDTO(BaseModel):
    id: Optional[int] = None
    titulo: str
    creado_en: datetime = datetime.now()

config = ProyectoConfig()
print(f"Iniciando arquitectura para: {config.nombre_app} v{config.version}")
```

⚠ Gotcha Frecuente

Colocar consultas SQL directamente dentro de los componentes visuales del frontend destruye la mantenibilidad.

❌ Antipatrón

En archivo del frontend: # cursor.execute('INSERT INTO...') ❌ Acoplamiento peligroso

✅ Patrón Correcto

Frontend -> Llama a API REST -> API invoca Repositorio -> BD ✅

Clase 02: Desarrollo del Backend: APIs RESTful con FastAPI

FastAPI es el framework web moderno de Python más rápido, diseñado para construir microservicios y APIs con tipado estricto.

☀ Metáfora: FastAPI como un Centro Logístico de Alta Velocidad para Peticiones HTTP

FastAPI es una ventanilla de atención ultra rápida: valida tu formulario antes de atenderte y te entrega un recibo oficial.

Principios Teóricos

Validación automática de requests y responses gracias a la integración profunda con Pydantic.

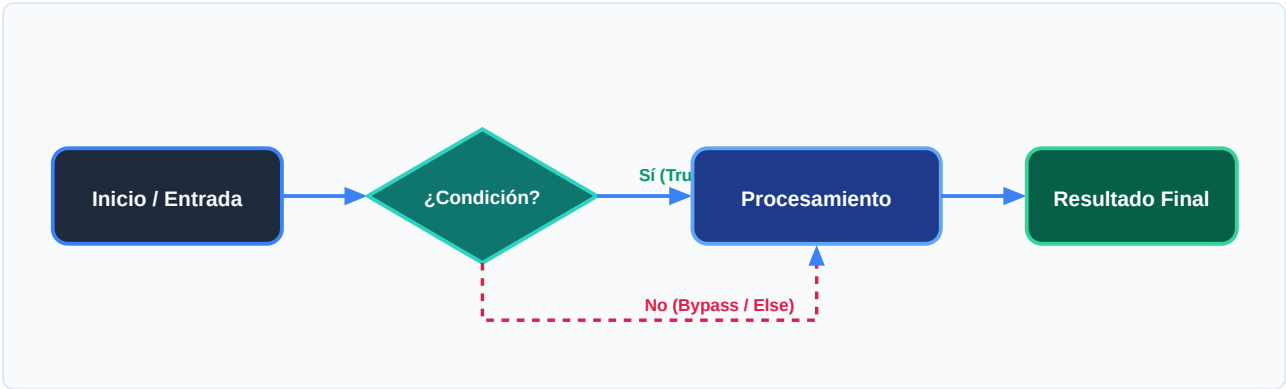
Generación automática de documentación interactiva en /docs (Swagger UI) y /redoc.

⚡ Regla de Oro

Retorna siempre códigos de estado HTTP semánticos (ej. 201 Created tras un POST exitoso).

Arquitectura y Movimiento de Datos

Flujo de una petición HTTP en FastAPI desde el router hasta la respuesta JSON.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Cliente envía HTTP Request (JSON body + headers).	Petición recibida en puerto 8000.
2. Proceso	Validación automática del esquema Pydantic.	DTO validado o 422 Unprocessable Entity.
3. Salida	Serialización del resultado y retorno de HTTP Response.	JSON emitido con código 200/201.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Servicio de Productos API", version="1.0.0")

class Producto(BaseModel):
    id: int
    nombre: str
    precio: float

DB_ITEMS = {}

@app.post("/productos", status_code=201)
def crear_producto(prod: Producto):
    if prod.id in DB_ITEMS:
        raise HTTPException(status_code=400, detail="El producto ya existe.")
    DB_ITEMS[prod.id] = prod
    return {"mensaje": "Creado con éxito", "producto": prod}

@app.get("/productos/{item_id}")
def obtener_producto(item_id: int):
    if item_id not in DB_ITEMS:
        raise HTTPException(status_code=404, detail="No encontrado")
    return DB_ITEMS[item_id]
```

⚠ Gotcha Frecuente

Usar funciones síncronas bloqueantes (como `time.sleep`) dentro de funciones `async def` congela todo el servidor.

❌ Antipatrón

`async def endpoint(): time.sleep(5) # ❌ Bloquea el event loop para todos los usuarios`

✅ Patrón Correcto

`async def endpoint(): await asyncio.sleep(5) # ✅ No bloqueante`

Clase 03: Persistencia de Datos: Modelado SQL y Transacciones ACID

La persistencia de datos garantiza que la información de los usuarios permanezca intacta tras apagar o reiniciar el servidor.

☀ Metáfora: La Base de Datos como una Bóveda Acorazada para la Información

Una transacción ACID es como una transferencia bancaria: o se descuenta de una cuenta y se acredita en la otra, o se cancela todo.

Principios Teóricos

Inyección SQL: La vulnerabilidad #1 de bases de datos. Ocurre al concatenar strings en consultas.

Consultas parametrizadas: Separan el código SQL de los datos proporcionados por el usuario.

⚡ Regla de Oro

NUNCA concatenes variables en consultas SQL; usa siempre placeholders (?, %s o :val).

Arquitectura y Movimiento de Datos

Ciclo de vida de una transacción con Commit / Rollback automático.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Apertura de conexión y bloque transaccional (with conn:).	Transacción iniciada en BD.
2. Proceso	Ejecución de sentencias DML (INSERT, UPDATE, DELETE).	Cambios en buffer transaccional.
3. Salida	Cierre seguro de la conexión.	Pool liberado.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
import sqlite3

class RepositorioUsuarios:
    def __init__(self, db_path: str = "app.db"):
        self.db_path = db_path
        self._crear_tabla()

    def _crear_tabla(self):
        with sqlite3.connect(self.db_path) as conn:
            conn.execute("""
                CREATE TABLE IF NOT EXISTS usuarios (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    nombre TEXT NOT NULL,
                    email TEXT UNIQUE NOT NULL
                )
            """)

    def insertar(self, nombre: str, email: str) -> int:
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.cursor()
            cursor.execute("INSERT INTO usuarios (nombre, email) VALUES (?, ?)", (nombre,
email))

            return cursor.lastrowid

repo = RepositorioUsuarios(":memory:")
uid = repo.insertar("Wisrovi Developer", "wisrovi@dev.com")
print(f"Usuario insertado con ID: {uid}")
```

⚠ Gotcha Frecuente

Formatear strings con f-strings en SQL permite a atacantes ejecutar comandos destructivos (ej. ' OR 1=1; DROP TABLE...).

❌ Antipatrón

```
cursor.execute(f'SELECT * FROM users WHERE email = \'{email}\')
```

 # ❌ Vulnerable a SQL Injection

✅ Patrón Correcto

```
cursor.execute('SELECT * FROM users WHERE email = ?', (email,))
```

 # ✅ 100% Seguro

Clase 04: Desarrollo del Frontend: Dashboards con Streamlit

Streamlit permite transformar scripts de Python en aplicaciones web interactivas para ciencia de datos e Inteligencia Artificial.

☀ Metáfora: Streamlit como el Salón de Control Visual para tu Backend de Python

Es como un tablero de mandos de automóvil donde cada botón y pantalla se conecta directamente al motor de tu backend.

Principios Teóricos

Modelo Reactivo: Cada vez que el usuario interactúa con un control (botón, slider), Streamlit reejecuta el script de arriba a abajo.

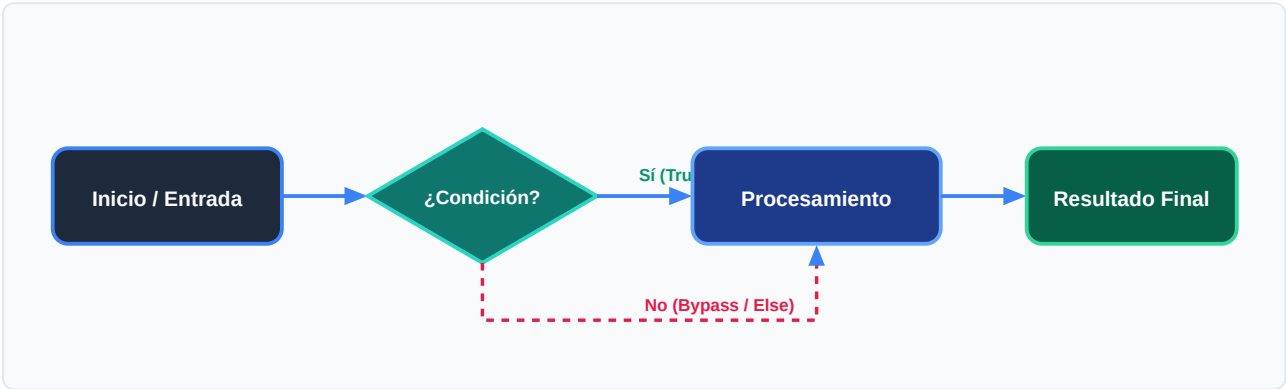
`st.session_state`: Permite preservar variables y estados entre reejecuciones del script.

⚡ Regla de Oro

Usa `st.session_state` para almacenar sesiones de chat o datos de formularios sin perderlos al hacer clic.

Arquitectura y Movimiento de Datos

Ciclo reactivo de eventos en Streamlit e invocación a la API.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Renderizado de componentes visuales (título, inputs).	DOM generado en el navegador.
2. Proceso	Interacción del usuario (clic en botón).	Evento disparado hacia el servidor.
3. Salida	Actualización de <code>st.session_state</code> y mensaje de éxito.	Interfaz actualizada instantáneamente.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
import streamlit as st

st.set_page_config(page_title="Panel de Control", page_icon="🚀")
st.title("🚀 Panel de Gestión de Leads")

if "leads" not in st.session_state:
    st.session_state.leads = []

with st.form("form_lead"):
    nombre = st.text_input("Nombre completo")
    email = st.text_input("Correo electrónico")
    enviado = st.form_submit_button("Guardar Lead")

    if enviado and nombre:
        st.session_state.leads.append({"nombre": nombre, "email": email})
        st.success(f"Lead {nombre} registrado con éxito.")

st.write(f"Total registrados: {len(st.session_state.leads)}")
```

⚠ Gotcha Frecuente

Cargar modelos pesados o archivos grandes en cada interacción ralentiza la aplicación.

❌ Antipatrón

`modelo = cargar_modelo_pesado_2gb()` # ❌ Se recarga en cada clic

✅ Patrón Correcto

`@st.cache_resource def get_model(): return cargar_modelo()` # ✅ Se ejecuta una sola vez en caché

Clase 05: Integración del Motor de IA y Agentes en la App

Integrar un agente en una aplicación web requiere gestionar latencias, streaming de texto y manejo de errores de API.

☀ Metáfora: Conectar el Cerebro del Agente al Sistema Nervioso de la Aplicación

Es como conectar un motor híbrido a un automóvil: debe responder con potencia suave sin tirones para el conductor.

Principios Teóricos

Streaming de respuestas: Enviar token por token al frontend para que el usuario no espere 10 segundos en blanco.

Manejo de rate limits: Reintentos exponenciales con backoff ante errores 429 de proveedores de IA.

⚡ Regla de Oro

Muestra siempre indicadores visuales de carga (spinners) mientras el agente razona.

Arquitectura y Movimiento de Datos

Flujo de streaming y comunicación de eventos agente-frontend.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Usuario envía mensaje en el chat del frontend.	Mensaje en cola de envío.
2. Proceso	Backend recibe solicitud y activa el bucle ReAct del agente.	Agente ejecutando herramientas.
3. Salida	Persistencia de la conversación en base de datos.	Historial actualizado.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
class AgenteService:
    def __init__(self, nombre_bot: str = "WisroviAssistant"):
        self.nombre_bot = nombre_bot

    def procesar_consulta(self, usuario_id: str, prompt: str) -> dict:
        # Lógica de agente con memoria y guardrails
        respuesta = f"[{self.nombre_bot}] He analizado tu solicitud: '{prompt}'. Todo en orden."
        return {
            "usuario_id": usuario_id,
            "respuesta": respuesta,
            "tokens_usados": 42
        }

servicio = AgenteService()
print(servicio.procesar_consulta("usr_1", "Generar balance"))
```

⚠ Gotcha Frecuente

Escribir las claves de API (OPENAI_API_KEY, GEMINI_API_KEY) en el código del frontend expone tu cuenta.

❌ Antipatrón

API_KEY = 'sk-123456789' # ❌ Expuesto en el repositorio

✅ Patrón Correcto

API_KEY = os.environ.get('GEMINI_API_KEY') # ✅
Variable de entorno segura

Clase 06: Testing Riguroso con Pytest, Mocks y Calidad

El testing automatizado es la única garantía de que los cambios nuevos no rompan funcionalidades existentes en producción.

☀ Metáfora: Los Tests como el Control de Calidad y Pruebas de Choque de un Vehículo

Hacer tests es como las pruebas de choque de los coches: verificas que los frenos funcionan antes de salir a la autopista.

Principios Teóricos

Mocks y Stubs: Simulan respuestas de servicios externos (como APIs de pago o LLMs) para tests rápidos y gratuitos.

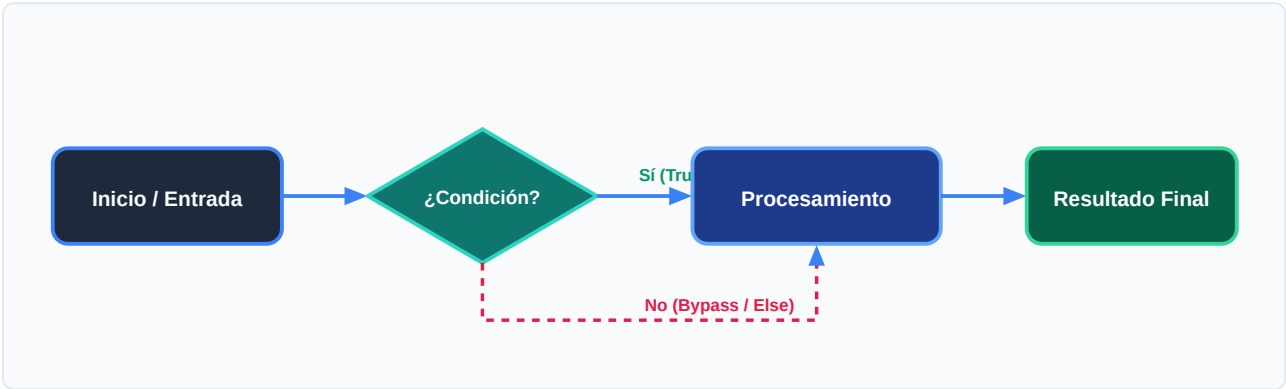
Fixtures de Pytest: Preparan el entorno (bases de datos temporales, clientes HTTP) antes de cada prueba.

⚡ Regla de Oro

Tus tests nunca deben depender de servicios externos reales ni requerir conexión a internet.

Arquitectura y Movimiento de Datos

Ejecución de suite de tests, fixtures y reporte de cobertura (Coverage).



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Descubrimiento de archivos test_*.py.	Tests recolectados por pytest.
2. Proceso	Inicialización de fixtures y mocks.	Entorno aislado preparado.
3. Salida	Tear-down y emisión de reporte verde (PASSED).	Suite completada.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
def calcular_subtotal(items: list[dict]) -> float:
    return sum(i["precio"] * i["cantidad"] for i in items)

def test_calculo_subtotal():
    carrito = [
        {"precio": 10.0, "cantidad": 2},
        {"precio": 5.0, "cantidad": 1}
    ]
    assert calcular_subtotal(carrito) == 25.0

def test_carrito_vacio():
    assert calcular_subtotal([]) == 0.0

print("Ejecutando tests...")
test_calculo_subtotal()
test_carrito_vacio()
print("✅ Todos los tests pasaron exitosamente.")
```

⚠️ Gotcha Frecuente

Hacer que los tests llamen a APIs reales falla si no hay internet y consume cuota de pago.

❌ Antipatrón

```
def test_llm(): res = llamar_api_real_openai() # ❌
Lento, frágil y cuesta dinero
```

✅ Patrón Correcto

```
def test_llm(mock): mock.patch('llm.call',
return_value='Respuesta Mock') # ✅ Rápido y
determinista
```

Clase 07: Containerización Profesional con Docker y Compose

Docker elimina el famoso problema de 'en mi máquina sí funciona' empaquetando el código con todas sus dependencias.

☀ Metáfora: Docker como Contenedores Estándar de Carga Marítima para Software

Un contenedor Docker es como un contenedor de barco: no importa si va en tren o camión, su contenido viaja aislado y seguro.

Principios Teóricos

Dockerfile: La receta paso a paso para construir la imagen del contenedor.

Docker Compose: La herramienta para definir y correr aplicaciones multi-contenedor con un solo comando.

⚡ Regla de Oro

Usa imágenes base ligeras (ej. `python:3.11-slim`) para reducir el tamaño y vulnerabilidades.

Arquitectura y Movimiento de Datos

Arquitectura de contenedores aislados comunicados por red interna.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Lectura de docker-compose.yml.	Servicios definidos.
2. Proceso	Build de imágenes para FastAPI y Streamlit.	Imágenes construidas localmente.
3. Salida	Vinculación de puertos (8000, 8501, 5432) hacia el host.	Aplicación disponible en localhost.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
DOCKERFILE_EXAMPLE = """FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]"""

print("Dockerfile de producción configurado:")
print(DOCKERFILE_EXAMPLE)
```

⚠ Gotcha Frecuente

Usar imágenes completas basadas en Ubuntu instala compiladores innecesarios generando imágenes de más de 2 GB.

❌ Antipatrón

FROM ubuntu:latest # ❌ Imagen pesada y lenta de descargar

✅ Patrón Correcto

FROM python:3.11-slim # ✅ Ligera (~150 MB) y rápida

Clase 08: Despliegue en la Nube, CI/CD y Portafolio Final

La graduación del programa culmina con el despliegue de tu solución y la consolidación de tu portafolio profesional.

☀ Metáfora: Lanzamiento a Producción y Presentación de tu Proyecto ante el Mundo

Es el corte de cinta inaugural de tu edificio de software: listo para recibir usuarios reales en todo el mundo.

Principios Teóricos

Pipelines de CI/CD: Automatizan la ejecución de tests y el despliegue automático tras cada git push.

Documentación README: La carta de presentación de tu proyecto para reclutadores y la comunidad.

⚡ Regla de Oro

Un proyecto sin README ni tests no está terminado; la excelencia de ingeniería se demuestra en los detalles.

Arquitectura y Movimiento de Datos

Flujo de integración continua: Git Push -> CI Runner -> Tests -> Deploy.



Fase	Acción del Intérprete	Estado Memoria
1. Inicio	Desarrollador hace push a la rama main.	Commit registrado en GitHub.
2. Proceso	GitHub Actions activa el runner ubuntu-latest.	Entorno virtual aprovisionado.
3. Salida	Despliegue a producción y notificación.	Aplicación en vivo y certificada.

Implementación en Python 3.10+

main.py (PEP 8)

Código Ejecutable

```
class ChecklistGraduacion:
    def __init__(self, autor: str, proyecto: str):
        self.autor = autor
        self.proyecto = proyecto
        self.items = {
            "1. Código modular y PEP 8": True,
            "2. Suite de pruebas con Pytest": True,
            "3. Dockerfile y Docker Compose": True,
            "4. Documentación README completa": True,
            "5. Video demo o capturas": True
        }

    def verificar(self) -> bool:
        return all(self.items.values())

grad = ChecklistGraduacion("Wisrovi Student", "AI Support Hub")
print(f"Estado de Graduación para {grad.autor}:")
for k, v in grad.items.items():
    print(f" [{ 'X' if v else ' ' }] {k}")
print(f"🏆 ¿Aprobado para Certificación?: {grad.verificar()}")
```

⚠ Gotcha Frecuente

Subir archivos temporales (__pycache__, .env, .venv) por no configurar un .gitignore limpio.

❌ Antipatrón

Repositorio con 100 archivos .pyc y credenciales
secretas ❌

✅ Patrón Correcto

Repositorio con .gitignore estándar de Python y
variables en secretos de GitHub ✅

Conclusiones Generales del Curso

Has completado las 8 semanas de **Curso 4: Taller Práctico & Proyecto Final Integrador**.



Certificación del Nivel

Dominio conceptual y práctico consolidado.

Bibliografía Canónica

Recurso	Descripción	Enlace
Documentación Oficial Python	Especificación canónica	docs.python.org/3/
PEP 8	Guía de estilo	peps.python.org/pep-0008/
wisrovi SUITE	Librerías open source	github.com/wisrovi